

Uma breve introdução ao Cálculo- λ

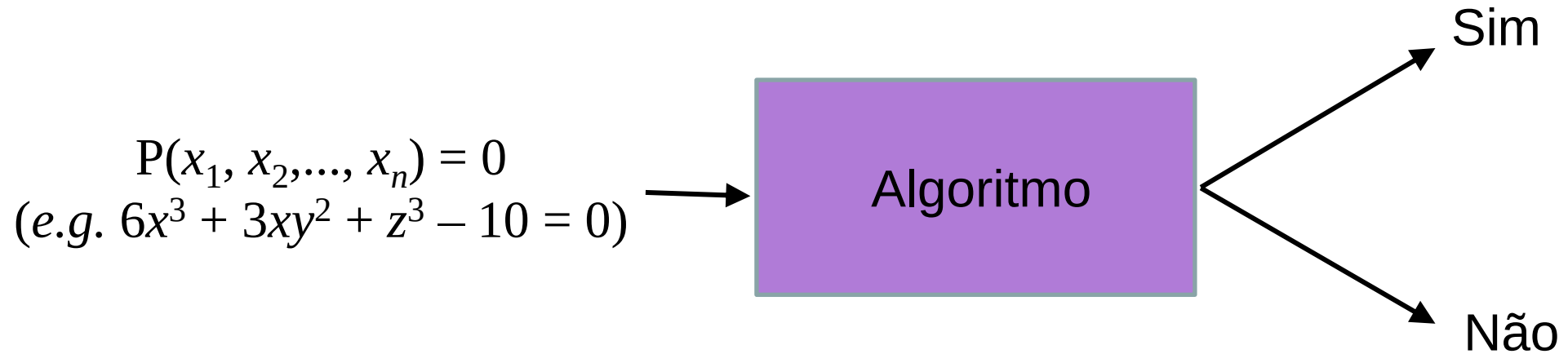
Problemas de Hilbert

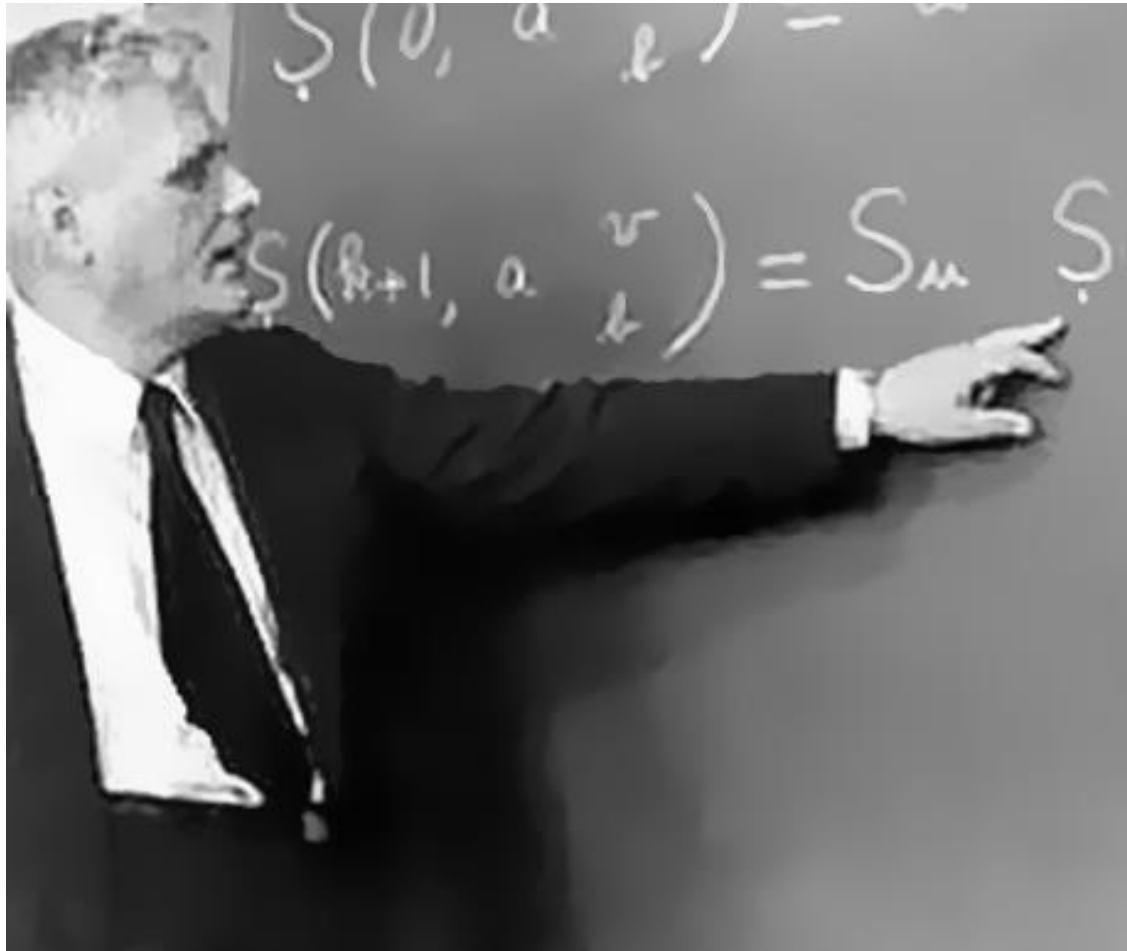
São uma lista de 23 problemas matemáticos propostos, como desafio para o século, por David Hilbert no **Congresso Internacional de Matemáticos** de 1900 em Paris. Na ocasião foram listados 10 problemas, a lista foi completada posteriormente.



10º Problema de Hilbert

Encontrar um procedimento (um algoritmo) para verificar se uma equação diofantina (uma equação polinomial com múltiplas variáveis e coeficientes inteiros) possui solução inteira.





AN UNSOLVABLE PROBLEM OF ELEMENTARY NUMBER THEORY.¹

By ALONZO CHURCH.

The purpose of the present paper is to propose a definition of effective calculability² which is thought to correspond satisfactorily to the somewhat vague intuitive notion in terms of which problems of this class are often stated, and to show, by means of an example, that not every problem of this class is solvable.

...

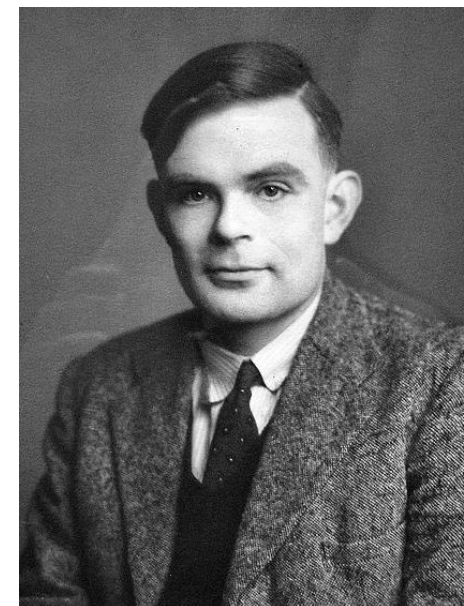
We introduce at once the following infinite list of abbreviations,

- $1 \rightarrow \lambda ab \cdot a(b),$
- $2 \rightarrow \lambda ab \cdot a(a(b)),$
- $3 \rightarrow \lambda ab \cdot a(a(a(b))),$

and so on, each positive integer in Arabic notation standing for a formula of the form $\lambda ab \cdot a(a(\cdots a(b) \cdots))$.



Kurt Gödel – Funções μ -recursivas



Alan Turing – Máquinas de Turing

Tese de Church-Turing

Uma função sobre números naturais pode ser calculada por um procedimento se e somente se ela pode ser computada por uma máquina de Turing.

Todas as funções computáveis são computáveis por uma máquina de Turing.

Por que estudar Cálculo- λ ?

1. Expressa qualquer computação possível.
2. É a fundamentação teórica sobre a qual as linguagens funcionais são projetadas.
3. Atualmente está disponível em boa parte das linguagens de programação.

Por que estudar Cálculo- λ ?

1. Expressa qualquer computação possível.
2. É a fundamentação teórica sobre a qual as linguagens funcionais são projetadas.
3. Atualmente está disponível em boa parte das linguagens de programação.

Por que estudar Cálculo- λ ?

1. Expressa qualquer computação possível.
2. É a fundamentação teórica sobre a qual as linguagens funcionais são projetadas.
3. Atualmente está disponível em boa parte das linguagens de programação.

Sintaxe de Cálculo- λ

$\langle \text{Expressão} \rangle \rightarrow x$ (uma variável a, b, c, ...)
| $(\langle \text{Expressão} \rangle \langle \text{Expressão} \rangle)$ (aplicação de função)
| $(\lambda x . \langle \text{Expressão} \rangle)$ (abstração λ)

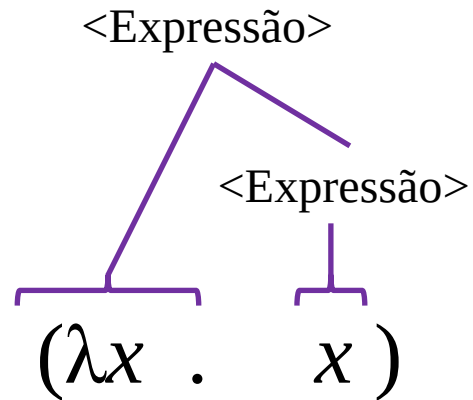
Exemplos de Expressões λ

$\langle \text{Expressão} \rangle \rightarrow x$ (uma variável a, b, c, ...)

$| (\langle \text{Expressão} \rangle \langle \text{Expressão} \rangle)$ (aplicação de função)

$| (\lambda x . \langle \text{Expressão} \rangle)$ (abstração λ)

$\lambda x . x$



$\langle \text{Expressão} \rangle$
 $(\lambda x . \langle \text{Expressão} \rangle)$
 $(\lambda x . x)$

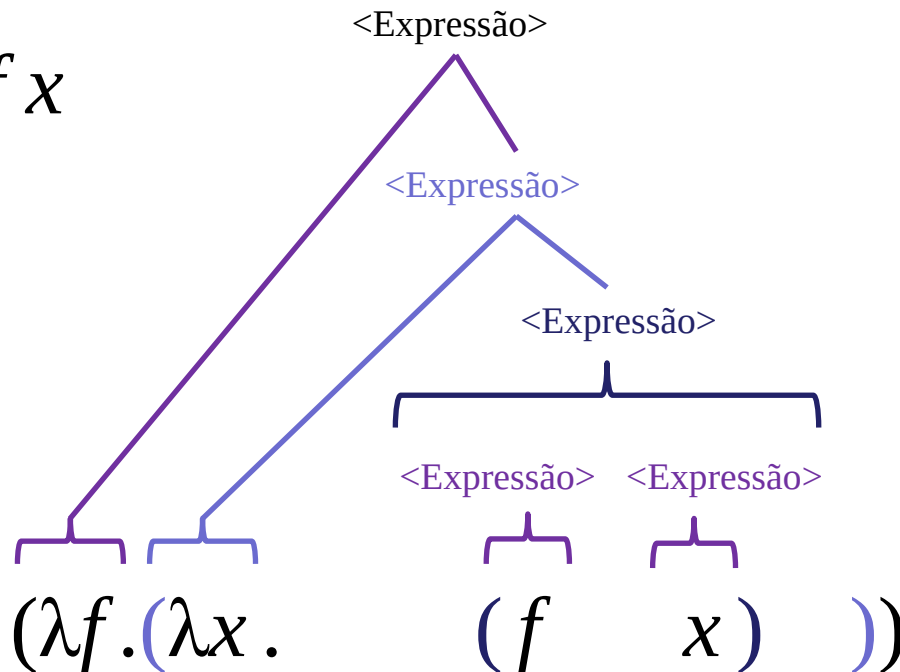
Exemplos de Expressões λ

$\langle \text{Expressão} \rangle \rightarrow x$ (uma variável a, b, c, ...)

 $| (\langle \text{Expressão} \rangle \langle \text{Expressão} \rangle)$ (aplicação de função)

 $| (\lambda x. \langle \text{Expressão} \rangle)$ (abstração λ)

$\lambda f. \lambda x. f x$



$\langle \text{Expressão} \rangle$

 $(\lambda f. \langle \text{Expressão} \rangle)$

 $(\lambda f. (\lambda x. \langle \text{Expressão} \rangle))$

 $(\lambda f. (\lambda x. (\langle \text{Expressão} \rangle \langle \text{Expressão} \rangle)))$

 $(\lambda f. (\lambda x. (f \langle \text{Expressão} \rangle)))$

 $(\lambda f. (\lambda x. (f x)))$

Eliminando alguns parênteses

$f\ g\ h\ x$

$((f\ g)\ h)\ x)$

Escopo

O escopo de uma variável é o contexto no qual a variável pode ser usada.
Em uma expressão $\lambda x.M$ a expressão M é o escopo da variável x .

$$(\lambda a. \lambda b. \lambda c. a(abc))(\lambda a. \lambda b. ab)$$

Parênteses e Escopo

$\lambda a \lambda b. b (\lambda x \lambda y. x) (\lambda s \lambda z. z)$ $(\lambda a \lambda b. b (\lambda x \lambda y. x) (\lambda s \lambda z. z))$

$(\lambda a \lambda b. b) (\lambda x \lambda y. x) (\lambda s \lambda z. z)$ $((\lambda a \lambda b. b) (\lambda x \lambda y. x)) (\lambda s \lambda z. z)$

$(\lambda a \lambda b. b)((\lambda x \lambda y. x)(\lambda s \lambda z. z))$

Equivalência α

Dizemos que duas expressões lambda são *α -equivalentes* caso difiram apenas nos nomes das variáveis que ocorrem ligadas as abstrações λ , funções *α -equivalentes* representam a mesma computação. Por exemplo:

$$\lambda x \lambda y. x \equiv_{\alpha} \lambda a \lambda b. a$$

$$\lambda f \lambda x. f \ x \equiv_{\alpha} \lambda a \lambda b. a \ b$$

Variáveis Livres e Ligadas

Uma variável livre é uma variável que não está ligada a uma abstração λ . O conjunto de variáveis livres de uma expressão M é representado por $FV(M)$, que é definido como:

$$\begin{aligned}FV(x) &= \{x\} \\FV(MN) &= FV(M) \cup FV(N) \\FV(\lambda x. M) &= FV(M) - \{x\}\end{aligned}$$

Se $FV(M) = \emptyset$ dizemos que M é um expressão fechada (também chamada de **combinador**).

Variáveis Livres e Ligadas

$$FV(x) = \{x\}$$

$$FV(MN) = FV(M) \cup FV(N)$$

$$FV(\lambda x. M) = FV(M) - \{x\}$$

$$FV(\lambda x. f x) = \{f\}$$

$$FV(\lambda x. f x) = FV(f x) - \{x\}$$

$$FV(f) \cup FV(x) - \{x\}$$

$$\{f\} \cup \{x\} - \{x\}$$

$$\{f\}$$

Variáveis Livres e Ligadas

$$FV(\lambda x \lambda f. g(fxy)) = \{g, y\}$$

Substituição e Redução β

A substituição de todas as ocorrências livres de uma variável x por um expressão M é representada por $[M/x]$. O axioma central de cálculo- λ envolve a substituição e é chamado de **redução β** . A *redução β* representa a aplicação de uma *abstração λ* (função) em um argumento:

$$(\lambda x. N)M \triangleright [M/x]N$$

Por exemplo: $(\lambda f. f x)(\lambda y. y) \triangleright (\lambda y. y) x \triangleright x$

Dizemos que essas expressões são ***β -equivalentes***.

Redução β

$(\lambda x. xx)(\lambda a. a)(\lambda b. \lambda c. c)$

$\triangleright (\lambda a. a)(\lambda a. a)(\lambda b. \lambda c. c)$

Redução β

$(\lambda x. xx)(\lambda a. a)(\lambda b. \lambda c. c)$

▷ $(\lambda a. a)(\lambda a. a)(\lambda b. \lambda c. c)$

▷ $(\lambda a. a)(\lambda b. \lambda c. c)$

Redução β

$(\lambda x. xx)(\lambda a. a)(\lambda b. \lambda c. c)$

▷ $(\lambda a. a)(\lambda a. a)(\lambda b. \lambda c. c)$

▷ $(\lambda a. a)(\lambda b. \lambda c. c)$

▷ $(\lambda b. \lambda c. c)$ ← **Forma Normal**

Uma expressão na qual não é possível aplicar uma redução β está na forma normal.

Numerais de Church

Os números naturais podem ser expressos por meio de 0 e seus sucessores, dessa forma o número 1 pode ser expresso como sucessor de zero, o 2 por sucessor do sucessor de zero e assim sucessivamente.

$$0 \equiv \lambda s \lambda z. z$$

$$1 \equiv \lambda s \lambda z. s(z)$$

$$2 \equiv \lambda s \lambda z. s(s(z))$$

$$3 \equiv \lambda s \lambda z. s(s(s(z)))$$

$$4 \equiv \lambda s \lambda z. s(s(s(s(z))))$$

...

$$SUC = \lambda w \lambda y \lambda x. y(wyx)$$

$$ADD = \lambda x \lambda y \lambda w \lambda u. x w (ywu)$$

Numerais de Church

SUC

2

$(\lambda w \lambda y \lambda x. y(wyx))(\lambda s \lambda z. s(s(z)))$

Numerais de Church

SUC

2

$(\lambda w \lambda y \lambda x. y(wyx))(\lambda s \lambda z. s(s(z)))$

Numerais de Church

SUC

2



$(\lambda w \lambda y \lambda x. y(wyx))(\lambda s \lambda z. s(s(z)))$

$\triangleright (\lambda y \lambda x. y((\lambda s \lambda z. s(s(z)))yx))$

Numerais de Church

SUC

2

$(\lambda w \lambda y \lambda x. y(wyx))(\lambda s \lambda z. s(s(z)))$

▷ $(\lambda y \lambda x. y((\lambda s \lambda z. s(s(z)))yx))$

Numerais de Church

SUC

2

$(\lambda w \lambda y \lambda x. y(wyx))(\lambda s \lambda z. s(s(z)))$

▷ $(\lambda y \lambda x. y((\lambda s \lambda z. s(s(z)))yx))$

▷ $(\lambda y \lambda x. y((\lambda z. y(y(z)))x))$

Numerais de Church

SUC

2

$(\lambda w \lambda y \lambda x. y(wyx))(\lambda s \lambda z. s(s(z)))$

▷ $(\lambda y \lambda x. y((\lambda s \lambda z. s(s(z)))yx))$

▷ $(\lambda y \lambda x. y((\lambda z. y(y(z)))x))$

Numerais de Church

SUC

2



$(\lambda w \lambda y \lambda x. y(wyx))(\lambda s \lambda z. s(s(z)))$

▷ $(\lambda y \lambda x. y((\lambda s \lambda z. s(s(z)))yx))$

▷ $(\lambda y \lambda x. y((\lambda z. y(y(z)))x))$

▷ $(\lambda y \lambda x. y(y(y(x))))$

Numerais de Church

SUC

2

$$(\lambda w \lambda y \lambda x. y(wyx))(\lambda s \lambda z. s(s(z)))$$

$$\triangleright (\lambda y \lambda x. y((\lambda s \lambda z. s(s(z)))yx))$$

$$\triangleright (\lambda y \lambda x. y((\lambda z. y(y(z)))x))$$

$$\triangleright (\lambda y \lambda x. y(y(y(x)))) \equiv_{\alpha} (\lambda s \lambda z. s(s(s(z))))$$

3

Booleanos

Valores booleanos, *verdade* e *falsidade*, podem ser representados pelas abstrações λ :

$$V \equiv \lambda a \lambda b. a$$

$$F \equiv \lambda a \lambda b. b$$

E os operadores lógicos definidos como:

$$E = \lambda x \lambda y. xy(\lambda u \lambda v. v)$$

$$OU = \lambda x \lambda y. x(\lambda u \lambda v. u)y$$

$$N\tilde{A}O = \lambda x. x(\lambda u \lambda v. v)(\lambda w \lambda z. w)$$

Booleanos

Se NÃO VERDADE então 1 senão 2

$$(\lambda x. x(\lambda u \lambda v. v)(\lambda w \lambda z. w))(\lambda a \lambda b. a)(\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$

Se **NÃO VERDADE** então **1** senão **2**

$(\lambda x. x(\lambda u \lambda v. v)(\lambda w \lambda z. w))(\lambda a \lambda b. a)(\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$

$\triangleright ((\lambda a \lambda b. a)(\lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$

Se **NÃO VERDADE** então **1** senão **2**

$$(\lambda x. x(\lambda u \lambda v. v)(\lambda w \lambda z. w))(\lambda a \lambda b. a)(\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright ((\lambda a \lambda b. a)(\lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright ((\lambda b \lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$

Se **NÃO VERDADE** então **1** senão **2**

$$(\lambda x. x(\lambda u \lambda v. v)(\lambda w \lambda z. w))(\lambda a \lambda b. a)(\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright ((\lambda a \lambda b. a)(\lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright ((\lambda b \lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright (\lambda u \lambda v. v) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$

Se **NÃO VERDADE** então **1** senão **2**

$$(\lambda x. x(\lambda u \lambda v. v)(\lambda w \lambda z. w))(\lambda a \lambda b. a)(\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright ((\lambda a \lambda b. a)(\lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright ((\lambda b \lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright (\lambda u \lambda v. v) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$$
$$\triangleright (\lambda v. v) (\lambda s \lambda z. (s(s(z))))$$

Se **NÃO VERDADE** então **1** senão **2**

$(\lambda x. x(\lambda u \lambda v. v)(\lambda w \lambda z. w))(\lambda a \lambda b. a)(\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$

▷ $((\lambda a \lambda b. a)(\lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$

▷ $((\lambda b \lambda u \lambda v. v)(\lambda w \lambda z. w)) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$

▷ $(\lambda u \lambda v. v) (\lambda s \lambda z. s(z))(\lambda s \lambda z. (s(s(z))))$

▷ $(\lambda v. v) (\lambda s \lambda z. (s(s(z))))$

▷ $\lambda s \lambda z. (s(s(z)))$

Cálculo- λ Enriquecido

Embora qualquer computação possa ser expressa por meio de cálculo- λ puro, o cálculo- λ enriquecido (com expressão condicional, algarismos arábicos, operadores aritméticos, operadores lógicos, operadores relacionais, construtores de dados, etc) é comumente usado. Uma vez que as expressões ficam muito mais concisas e consideravelmente mais simples.

$\lambda x. x + 1$ *successor*

$\lambda x \lambda y. \text{if } x < y \text{ then } x \text{ else } y$ *maior*

Curificação (*Currying*)

Funções com múltiplos parâmetros, em cálculo- λ , são geralmente tratadas de forma linearizada (currificada) como múltiplas aplicações de funções com um único parâmetro. Por exemplo, uma função com dois parâmetros, $\lambda x \lambda y. M$, é avaliada como uma função que recebe um parâmetro formal x e retorna uma função que recebe um parâmetro formal y .

$$\begin{aligned} & (\lambda x \lambda y. x + y) \ 5 \ 2 \\ & \quad \underbrace{\lambda x \lambda y. x + y}_{\lambda y. 5 + y} \quad \swarrow \\ & \triangleright (\lambda y. 5 + y) \ 2 \\ & \quad \underbrace{\lambda y. 5 + y}_{5 + 2} \\ & \triangleright (5 + 2) \triangleright 7 \end{aligned}$$

Curificação (*Currying*)



A aplicação parcial de argumentos é conhecida como curificação em homenagem ao matemático **Haskell Curry** que investigou essa propriedade do cálculo- λ , inspirado no trabalho de Moses Schönfinkel.